EVM 프리컴파일 컨트랙트와 최적화 분석

개념적 배경

프리컴파일 컨트랙트(Precompiled Contracts)는 EVM에서 네이티브 Go 코드로 구현된 특수한 컨트랙트입니다. 복잡한 암호학적 연산이나 자주 사용되는 기능을 효율적으로 제공하기 위해 특정 주소에 미리 배치되어 있습니다.

설계 철학

1. **성능 최적화**: 복잡한 연산을 네이티브 코드로 구현하여 속도 향상
2. **가스 효율성**: EVM 바이트코드보다 훨씬 저렴한 비용으로 제공
3. **표준화**: 공통적으로 필요한 기능의 일관된 구현
4. **확장성**: 하드포크를 통한 새로운 기능 추가

프리컴파일 컨트랙트의 장점

1. **속도**: 네이티브 코드 실행으로 수십 배 빠른 성능
2. **가스 절약**: 동일한 기능을 EVM에서 구현할 때보다 저렴
3. **보안**: 검증된 암호학 라이브러리 사용
4. **일관성**: 모든 클라이언트에서 동일한 구현

구현 분석

프리컴파일 인터페이스

모든 프리컴파일 컨트랙트는 공통 인터페이스를 구현합니다:

```
type PrecompiledContract interface {
RequiredGas(input []byte) uint64  // 가스 비용 계산
Run(input []byte) ([]byte, error) // 실제 실행
}
```

하드포크별 프리컴파일 세트

Homestead (초기 세트)

```
var  PrecompiledContractsHomestead = PrecompiledContracts{
common.BytesToAddress([]byte{0x1}): &ecrecover{}, // ECDSA 복구
common.BytesToAddress([]byte{0x2}): &sha256hash{}, // SHA-256 해시
common.BytesToAddress([]byte{0x3}): &ripemd160hash{}, // RIPEMD-160 해시
common.BytesToAddress([]byte{0x4}): &dataCopy{}, // 데이터 복사
}
```

Byzantium (타원곡선 암호 추가)

```
var  PrecompiledContractsByzantium = PrecompiledContracts{
// ... Homestead 컨트랙트들
common.BytesToAddress([]byte{0x5}): &bigModExp{}, // 모듈러 지수
common.BytesToAddress([]byte{0x6}): &bn256AddByzantium{}, // BN256 점 덧셈
common.BytesToAddress([]byte{0x7}): &bn256ScalarMulByzantium{}, // BN256 스칼라 곱
common.BytesToAddress([]byte{0x8}): &bn256PairingByzantium{}, // BN256 페어링
```

```
}
```

Istanbul (성능 개선)

```
var PrecompiledContractsIstanbul = PrecompiledContracts{
// ... 기존 컨트랙트들 (BN256 가스 비용 최적화)
common.BytesToAddress([]byte{0x9}): &blake2F{}, // BLAKE2F 해시
}
```

Cancun (KZG 및 임시 스토리지)

```
var PrecompiledContractsCancun = PrecompiledContracts{
// ... 기존 컨트랙트들
common.BytesToAddress([]byte{0xa}): &kzgPointEvaluation{}, // KZG 점 평가
}
```

Prague (BLS12-381 곡선)

```
var PrecompiledContractsPrague = PrecompiledContracts{
// ... 기존 컨트랙트들
common.BytesToAddress([]byte{0x0b}): &bls12381G1Add{}, // BLS12-381 G1 덧셈
common.BytesToAddress([]byte{0x0c}): &bls12381G1MultiExp{}, // BLS12-381 G1 다중
common.BytesToAddress([]byte{0x0d}): &bls12381G2Add{}, // BLS12-381 G2 덧셈
common.BytesToAddress([]byte{0x0e}): &bls12381G2MultiExp{}, // BLS12-381 G2 다중
```

```
common.BytesToAddress([]byte{0x0f}): &bls12381Pairing{}, // BLS12-381 페어링
common.BytesToAddress([]byte{0x10}): &bls12381MapG1{}, // BLS12-381 G1 매핑
common.BytesToAddress([]byte{0x11}): &bls12381MapG2{}, // BLS12-381 G2 매핑
}
```

주요 프리컴파일 컨트랙트 분석

1. ECRECOVER (0x01) - ECDSA 서명 복구

```
type ecrecover struct{}

func (c *ecrecover) RequiredGas(input []byte) uint64 {
return  params.EcrecoverGas  // 3000 가스
}

func (c *ecrecover) Run(input []byte) ([]byte, error) {
const  ecRecoverInputLength = 128
input = common.RightPadBytes(input, ecRecoverInputLength)
// 입력: (hash, v, r, s), 각각 32바이트
r := new(big.Int).SetBytes(input[64:96])
s := new(big.Int).SetBytes(input[96:128])
v := input[63] - 27
// 서명 유효성 검증
if !allZero(input[32:63]) || !crypto.ValidateSignatureValues(v, r, s, false) {
```

```
return nil, nil
}
// 서명 형식 변환 (v를 끝으로 이동)
sig := make([]byte, 65)
copy(sig, input[64:128])
sig[64] = v
// 공개키 복구
pubKey, err := crypto.Ecrecover(input[:32], sig)
if err != nil {
return nil, nil
}
// 주소 계산 (공개키의 Keccak256 해시의 마지막 20바이트)
return common.LeftPadBytes(crypto.Keccak256(pubKey[1:])[12:], 32), nil
}
```

최적화 기법:

- 네이티브 secp256k1 라이브러리 사용
- 입력 검증을 통한 조기 실패 처리
- 메모리 할당 최소화

2. SHA256 (0x02) - SHA-256 해시

```
type sha256hash struct{}
```

```
func (c *sha256hash) RequiredGas(input []byte) uint64 {
// 기본 60 가스 + 워드당 12 가스
return uint64(len(input)+31)/32*params.Sha256PerWordGas + params.Sha256BaseGas
}

func (c *sha256hash) Run(input []byte) ([]byte, error) {
h := sha256.Sum256(input) // Go 표준 라이브러리 사용
return h[:], nil
}
```

최적화 기법:

- Go 표준 라이브러리의 최적화된 구현 활용
- 하드웨어 가속 지원 (AES-NI 등)
- 단순한 인터페이스로 오버헤드 최소화

3. BIGMODEXP (0x05) - 모듈러 지수 연산

```
type bigModExp struct {
eip2565 bool // Berlin 하드포크 최적화
eip7823 bool // 입력 크기 제한
eip7883 bool // 추가 최적화
}

func (c *bigModExp) RequiredGas(input []byte) uint64 {
var (
```

```
baseLen = new(big.Int).SetBytes(getData(input, 0, 32))

expLen = new(big.Int).SetBytes(getData(input, 32, 32))

modLen = new(big.Int).SetBytes(getData(input, 64, 32))

)

// 조정된 지수 길이 계산

var  expHead *big.Int

if  big.NewInt(int64(len(input))).Cmp(baseLen) <= 0 {

expHead = new(big.Int)

} else {

if  expLen.Cmp(big32) > 0 {

expHead = new(big.Int).SetBytes(getData(input, baseLen.Uint64(), 32))

} else {

expHead = new(big.Int).SetBytes(getData(input, baseLen.Uint64(), expLen.Uint64()

}

}

// MSB 계산

var  msb  int

if  bitlen := expHead.BitLen();  bitlen > 0 {

msb = bitlen - 1

}

adjExpLen := new(big.Int)

if  expLen.Cmp(big32) > 0 {

adjExpLen.Sub(expLen, big32)
```

```
if c.eip7883 {
adjExpLen.Lsh(adjExpLen, 4) // * 16
} else {
adjExpLen.Lsh(adjExpLen, 3) // * 8
}
}
adjExpLen.Add(adjExpLen, big.NewInt(int64(msb)))
// 가스 비용 계산
gas := new(big.Int)
if modLen.Cmp(baseLen) < 0 {
gas.Set(baseLen)
} else {
gas.Set(modLen)
}
if c.eip2565 {
// EIP-2565 최적화된 공식
gas.Add(gas, big7)
gas.Rsh(gas, 3) // / 8
gas.Mul(gas, gas) // ^ 2
var minPrice uint64 = 200
if c.eip7883 {
minPrice = 500
if gas.Cmp(big32) > 0 {
```

```
gas.Add(gas, gas) // * 2
} else {
gas = big.NewInt(16)
}
}
if  adjExpLen.Cmp(big1) > 0 {
gas.Mul(gas, adjExpLen)
}
if !c.eip7883 {
gas.Div(gas, big3) // / 3
}
if  gas.BitLen() > 64 {
return  math.MaxUint64
}
return  max(minPrice, gas.Uint64())
}
// 이전 공식 (더 비쌈)
gas = modexpMultComplexity(gas)
if  adjExpLen.Cmp(big1) > 0 {
gas.Mul(gas, adjExpLen)
}
gas.Div(gas, big20)
if  gas.BitLen() > 64 {
```

```
return math.MaxUint64
}
return gas.Uint64()
}


func (c *bigModExp) Run(input []byte) ([]byte, error) {
var (
baseLen = new(big.Int).SetBytes(getData(input, 0, 32)).Uint64()
expLen = new(big.Int).SetBytes(getData(input, 32, 32)).Uint64()
modLen = new(big.Int).SetBytes(getData(input, 64, 32)).Uint64()
)
// 크기 제한 (EIP-7823)
if c.eip7823 && max(baseLen, expLen, modLen) > 1024 {
return nil, fmt.Errorf("input length exceeded 1024 bytes")
}
var (
base = new(big.Int).SetBytes(getData(input, 96, baseLen))
exp = new(big.Int).SetBytes(getData(input, 96+baseLen, expLen))
mod = new(big.Int).SetBytes(getData(input, 96+baseLen+expLen, modLen))
v []byte
)
switch {
case mod.BitLen() == 0:
```

```
// 모듈로 0은 정의되지 않음
return common.LeftPadBytes([]byte{}, int(modLen)), nil
case base.BitLen() == 1:
// base == 1인 경우 최적화
v = base.Mod(base, mod).Bytes()
default:
v = base.Exp(base, exp, mod).Bytes() // Go의 최적화된 구현
}
return common.LeftPadBytes(v, int(modLen)), nil
}
```

최적화 기법:

- EIP-2565: 가스 비용 공식 개선으로 70% 비용 절감
- 특수 케이스 최적화 (base == 1)
- Go의 big.Int 최적화된 지수 연산 활용
- 입력 크기 제한으로 DoS 방지

4. BN256 타원곡선 연산 (0x06-0x08)

```
// BN256 점 덧셈
type bn256AddIstanbul struct{}

func (c *bn256AddIstanbul) RequiredGas(input []byte) uint64 {
return params.Bn256AddGasIstanbul // 150 가스 (Byzantium: 500)
```

```
}


func (c *bn256AddIstanbul) Run(input []byte) ([]byte, error) {
return runBn256Add(input)
}


func runBn256Add(input []byte) ([]byte, error) {
x, err := newCurvePoint(getData(input, 0, 64))
if err != nil {
return nil, err
}
y, err := newCurvePoint(getData(input, 64, 64))
if err != nil {
return nil, err
}
res := new(bn256.G1)
res.Add(x, y) // 최적화된 타원곡선 덧셈
return res.Marshal(), nil
}
```

최적화 기법:

- Istanbul 하드포크에서 가스 비용 대폭 절감
- 전용 타원곡선 라이브러리 사용

- 하드웨어 가속 지원

5. BLAKE2F (0x09) - BLAKE2b 압축 함수

```
type blake2F struct{}

func (c *blake2F) RequiredGas(input []byte) uint64 {
// 라운드 수에 비례한 가스 비용
if len(input) < 4 {
return 0
}
return uint64(binary.BigEndian.Uint32(input[0:4]))
}

func (c *blake2F) Run(input []byte) ([]byte, error) {
// BLAKE2b 압축 함수 실행
return blake2b.F(input)
}
```

프리컴파일 실행 메커니즘

프리컴파일 호출 과정

```
func RunPrecompiledContract(p PrecompiledContract, input []byte, suppliedGas u
// 1. 가스 비용 계산
```

```
gasCost := p.RequiredGas(input)

if suppliedGas < gasCost {

return nil, 0, ErrOutOfGas

}

// 2. 가스 차감 로깅

if logger != nil && logger.OnGasChange != nil {

logger.OnGasChange(suppliedGas, suppliedGas-gasCost, tracing.GasChangeCallPrecom

}

suppliedGas -= gasCost

// 3. 실제 실행

output, err := p.Run(input)

return output, suppliedGas, err

}
```

하드포크별 활성화

```
func activePrecompiledContracts(rules params.Rules) PrecompiledContracts {

switch {

case rules.IsOsaka:

return PrecompiledContractsOsaka

case rules.IsPrague:

return PrecompiledContractsPrague

case rules.IsCancun:
```

```
return PrecompiledContractsCancun
case rules.IsBerlin:
return PrecompiledContractsBerlin
case rules.IsIstanbul:
return PrecompiledContractsIstanbul
case rules.IsByzantium:
return PrecompiledContractsByzantium
default:
return PrecompiledContractsHomestead
}
}
```

인터프리터 최적화 기법

1. 점프 테이블 최적화

직접 배열 접근

```
type JumpTable [256]*operation

// O(1) 시간에 operation 조회
operation := jumpTable[opcode]
```

함수 포인터 사용

```
type executionFunc func(pc *uint64, interpreter *EVMInterpreter, callContext *Sc


// 가상 함수 호출 오버헤드 최소화
res, err = operation.execute(&pc, in, callContext)
```

2. 메모리 풀링

스택 풀링

```
var stackPool = sync.Pool{
New: func() interface{} {
return &Stack{data: make([]uint256.Int, 0, 16)}
},
}


// 스택 재사용으로 GC 압박 감소
func newstack() *Stack {
return stackPool.Get().(*Stack)
}


func returnStack(s *Stack) {
s.data = s.data[:0] // 길이만 0으로 설정
stackPool.Put(s)
```

```
}
```

메모리 풀링

```
var memoryPool = sync.Pool{
New: func() any {
return &Memory{}
},
}

func (m *Memory) Free() {
// 16KB 이하만 풀에 반환 (메모리 사용량 제한)
const maxBufferSize = 16 << 10
if cap(m.store) <= maxBufferSize {
m.store = m.store[:0]
m.lastGasCost = 0
memoryPool.Put(m)
}
}
```

3. 해시 인스턴스 재사용

```
type EVMInterpreter struct {
hasher crypto.KeccakState // 재사용되는 해시 인스턴스
```

```
hasherBuf common.Hash // 재사용되는 결과 버퍼
}

func opKeccak256(pc *uint64, interpreter *EVMInterpreter, scope *ScopeContext)
// 해시 인스턴스 재사용
interpreter.hasher.Reset()
interpreter.hasher.Write(data)
interpreter.hasher.Read(interpreter.hasherBuf[:])
size.SetBytes(interpreter.hasherBuf[:])
return nil, nil
}
```

4. 스택 연산 최적화

In-place 연산

```
// 효율적: 결과를 기존 스택 위치에 저장
func opAdd(pc *uint64, interpreter *EVMInterpreter, scope *ScopeContext) ([]byt
x, y := scope.Stack.pop(), scope.Stack.peek() // y는 참조
y.Add(&x, y) // 결과를 y에 직접 저장
return nil, nil
}
```

불필요한 복사 방지

```
// 메모리 포인터 직접 반환
func (m *Memory) GetPtr(offset, size uint64) []byte {
return m.store[offset : offset+size]
}
```

5. 가스 계산 최적화

상수 가스와 동적 가스 분리

```
type operation struct {
constantGas uint64 // 컴파일 타임에 결정
dynamicGas gasFunc // 런타임에 계산
}

// 상수 가스는 즉시 차감
cost = operation.constantGas
contract.Gas -= cost

// 동적 가스는 필요시에만 계산
if operation.dynamicGas != nil {
dynamicCost, err := operation.dynamicGas(evm, contract, stack, mem, memorySize)
contract.Gas -= dynamicCost
```

```
}
```

성능 벤치마크

프리컴파일 vs EVM 구현 비교

| 연산 | EVM 가스 | 프리컴파일 가스 | 성능 향상 |

|------|----------|----------|----------|

| SHA256 (32바이트) | ~1000 | 72 | 14배 |

| ECRECOVER | ~30000 | 3000 | 10배 |

| MODEXP (256비트) | ~50000 | 1000 | 50배 |

| BN256 ADD | ~28000 | 150 | 187배 |

최적화 효과

EIP-2565 MODEXP 최적화

```
이전: gas = multComplexity(max(baseLen, modLen)) * max(adjExpLen, 1) / 20

이후: gas = ceiling(max(baseLen, modLen) / 8)^2 * max(adjExpLen, 1) / 3


평균 70% 가스 절감
```

Istanbul BN256 최적화

```
BN256 ADD: 500 → 150 가스 (70% 절감)

BN256 MUL: 40000 → 6000 가스 (85% 절감)
```

```
BN256 PAIRING: 100000 + 80000*k → 45000 + 34000*k 가스 (55% 절감)
```

실제 사용 예시

ECRECOVER 사용 (서명 검증)

```
function verifySignature(bytes32 hash, uint8 v, bytes32 r, bytes32 s) public p
return ecrecover(hash, v, r, s); // 0x01 프리컴파일 호출
}
```

SHA256 사용 (해시 계산)

```
function hashData(bytes memory data) public pure returns (bytes32) {
return sha256(data); // 0x02 프리컴파일 호출
}
```

BN256 사용 (zk-SNARK 검증)

```
function verifyProof(uint256[8] memory proof) public view returns (bool) {
// BN256 페어링을 사용한 영지식 증명 검증
// 0x08 프리컴파일 호출
}
```

관련 시스템

의존성 관계

- **EVM**: 프리컴파일 호출 및 가스 관리
- **StateDB**: 계정 존재 확인
- **ChainRules**: 하드포크별 활성화 결정
- **Crypto Libraries**: 암호학적 연산 구현

상호작용 패턴

1. **EVM → Precompile**: CALL 연산을 통한 호출
2. **Precompile → Crypto**: 네이티브 암호학 라이브러리 사용
3. **Precompile → Gas**: 입력 크기 기반 가스 계산
4. **ChainRules → Precompile**: 하드포크별 기능 활성화

영향 범위

- **DApp 성능**: 암호학적 연산의 효율성
- **가스 비용**: 복잡한 연산의 경제성
- **보안**: 검증된 암호학 구현
- **호환성**: 하드포크 간 일관성

이 프리컴파일 시스템은 Ethereum의 성능과 기능성을 크게 향상시키는 핵심 구성 요소로, 지속적인 최적화와 새로운 기능 추가를 통해 발전하고 있습니다.